

Plan de Investigación en Pregrado

Entrenamiento por sueño/vigilia y aprendizaje composicional

MAT2320 — Guía de trabajo semestral

0. Cómo leer este documento

Este plan tiene dos niveles:

1. **Plan general del semestre** (16 semanas, nivel superficial): te da la dirección global, para que sepas hacia dónde vas incluso cuando estés metido en el detalle de una semana particular.
2. **Plan particular de las semanas 1-3**: concreto, con tareas, horas estimadas y lecturas específicas.

Está pensado para una carga de **6-9 horas semanales** (sin contar la reunión con tu profesor ni la redacción del reporte semanal, que deberías presupuestar aparte, ~1 hora).

No lo sigas de forma rígida: es una guía razonable, no un contrato. Si una semana te toma más leer algo porque es denso, está bien — de hecho es esperable, ya que estás entrando a un área con bastante aparataje matemático nuevo (computabilidad, complejidad de Kolmogorov, inferencia variacional). Lo importante es que *cada semana termines con algo escrito* (tu resumen de avances/dudas/propuesta), no que hayas "completado" una lista.

1. Diagnóstico rápido: qué necesitas y qué no

Con RL + Deep Learning y PAC/online learning ya tienes:

- Redes neuronales, entrenamiento por gradiente, nociones de generalización.
- PAC learning, VC-dimension (esto es exactamente lo que se usa en la Sección 4.5 de las notas de composicionalidad — no necesitas repasarlo desde cero, solo tenerlo fresco).
- Intuición de MDPs / aprendizaje por refuerzo, útil para entender el paper de Dreamer (Documento 2) sin partir de cero.

Lo que **no** tienes y sí vas a necesitar, en orden de urgencia:

Tema	¿Por qué lo necesitas?	¿Cuánto necesitas realmente?
Máquinas de Turing / computabilidad	Para que la Definición 2.1 (complejidad de Kolmogorov) y el Teorema 2.4 (no-computabilidad de K) tengan sentido, no como fórmulas sino como objetos que entiendes	Poco: lo justo para saber qué es una máquina universal, qué es "computable", y por qué el problema de la parada implica que K no se puede calcular. No necesitas un curso completo de teoría de la computación.
Complejidad de Kolmogorov y MDL	Es el lenguaje común de los tres documentos : es literalmente el criterio que DreamCoder optimiza (Ecuación 1 de las notas de DreamCoder)	Medio-alto: esto sí es central a tu tema, no un prerrequisito lateral.
Inferencia variacional / ELBO / modelo de Helmholtz	El algoritmo wake-sleep de DreamCoder está formalmente emparentado con esto (Sección 5 de las notas de DreamCoder)	Medio: no necesitas dominar VAEs a fondo, pero sí entender qué es un ELBO y por qué wake-sleep es "el primo discreto y jerárquico" de una VAE.
Gramáticas probabilísticas / lambda cálculo tipado	Es el objeto matemático que es la librería L en DreamCoder (Sección 4 de esas notas)	Bajo: las notas ya son bastante autocontenidas en esto; no hace falta un libro aparte salvo que te bloquee.
Compositional generalization (benchmarks y definiciones)	Es la otra mitad del título de tu investigación	Alto: tan central como Kolmogorov/MDL.

2. Plan general del semestre (16 semanas) — nivel superficial

Fase	Semanas	Objetivo central	Output esperado
A. Fundamentos	1-3	Vocabulario común: Kolmogorov/MDL, wake-sleep clásico, DreamCoder, definiciones de composicionalidad	3 reportes semanales; mapa conceptual propio de cómo se conectan los 3 documentos
B. Profundización técnica	4-6	Entender wake-sleep/DreamCoder al nivel de poder explicarlo sin las notas delante; conexión con inferencia variacional	Resumen técnico propio (no solo repetir las notas) de la Sección 3 y 5 de las notas de DreamCoder
C. Revisión de literatura dirigida	7-9	Salir de los 3 documentos-semilla y mapear qué mecanismos "sueño/vigilia" existen <i>hoy</i> en ML (más allá de DreamCoder) y qué evidencia hay de que ayudan a composicionalidad	Bibliografía anotada (tabla comparativa de ~8-12 papers)
D. Síntesis y pregunta de investigación	10-12	Aterrizar una pregunta específica y acotada; opcionalmente correr un experimento pequeño/reproducir algo de DreamCoder en un dominio juguete	Borrador de "Related Work" + pregunta de investigación formulada con precisión
E. Redacción y cierre	13-16	Informe final; dejar una base clara para un posible paper el próximo semestre	Informe final + (idealmente) sección de "trabajo futuro" concreta

Nota sobre la Fase D (experimento): no es obligatorio que corras código — el objetivo del semestre, según el correo de tu profesor, es sobre todo teórico/de revisión ("bases para un paper"). Pero si en la Fase B ves que el código de DreamCoder es reproducible (Ellis et al. lo liberaron open-source) y te queda tiempo, correr un ejemplo de juguete (p. ej. list processing) te dará intuición que ninguna lectura te va a dar. Trátalo como *stretch goal*, no como requisito de la Semana X.

3. Plan particular — Semanas 1, 2 y 3

Semana 1 — Entrar al marco teórico (Kolmogorov/MDL) y al problema

Objetivo de la semana: que la Sección 2 de "Aprendizaje Composicional y Generalización OOD" (Documento 1) deje de sentirse como fórmulas sueltas y se sienta como una idea que puedes explicar con tus propias palabras.

Tarea	Tiempo aprox.	Detalle
Leer completo el Documento 1 (notas de composicionalidad), primera pasada	2.5 h	No te detengas en cada demostración. Primera pasada = entender la estructura: qué problema se plantea (Sec. 1), qué marco teórico se usa (Sec. 2), qué benchmarks existen (Sec. 3), qué síntomas se definen (Sec. 4). Anota qué NO entendiste, sin resolverlo todavía.
Repaso de computabilidad — lo mínimo	1.5-2 h	Ver sección 5 de este documento más abajo para la fuente recomendada. Objetivo único: entender la Definición 2.1 y el Teorema 2.4 del Documento 1 (por qué $K(x)$ no es computable) a nivel intuitivo.
Segunda pasada sobre Sección 2 del Documento 1 (Kolmogorov + MDL), con lápiz	1.5 h	Ahora sí, trabaja las definiciones 2.1, 2.6, 2.8 y el Ejemplo 2.2 con calma. Trata de reproducir el Ejemplo 2.2 sin mirar.
Primera lectura del Documento 3 (DreamCoder), Secciones 1-2 solamente	1.5 h	Fíjate en la Ecuación (1) de esas notas — es la misma idea de MDL del Documento 1, aplicada a programas. Esa es la conexión que sostiene todo tu tema.
Escribir el reporte semanal 1	1 h	Ver plantilla en la Sección 6.

Total aprox.: 8-8.5 h

Pregunta guía para cerrar la semana: ¿podrías explicarle a un compañero de otra carrera, en 3 frases, qué es el principio MDL y por qué "una hipótesis simple que explica bien los datos" es una forma de comprimir información?

Semana 2 — Wake-sleep y DreamCoder en profundidad

Objetivo de la semana: entender el algoritmo wake-sleep de DreamCoder al nivel de las tres fases (Sec. 3 del Documento 3) y su relación con inferencia variacional (Sec. 5).

Tarea	Tiempo aprox.	Detalle
Documento 3, Secciones 3 y 4 completas	2 h	Presta especial atención a la Ecuación (7) (score de búsqueda) y la Ecuación (9) (ganancia de compresión Δg) — son el corazón algorítmico del sistema.
Documento 3, Sección 5 (conexión con Helmholtz machine / VAE)	1 h	No necesitas dominar VAEs; basta con entender la Tabla 1 (la correspondencia) y por qué DreamCoder es "wake-sleep discreto y jerárquico".
Leer el paper original: Hinton, Dayan, Frey & Neal (1995) , " <i>The wake-sleep algorithm for unsupervised neural networks</i> ", Science	1.5 h	Es corto (4 páginas) y es la fuente primaria del nombre de tu proyecto. Vale la pena leerlo directo, no solo vía las notas.
Leer selectivamente el paper original de DreamCoder: Ellis et al. (2021) , PLDI (arxiv.org/abs/2006.08381)	2–2.5 h	No lo leas entero de una sentada. Lee: abstract, introducción, Sección 2 (arquitectura), y la sección de resultados/ablações. Salta el detalle de implementación de bajo nivel por ahora. <i>Alternativa más liviana</i> : hay una versión de revista, más corta y con más figuras: Ellis et al., " DreamCoder: growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning ", <i>Phil. Trans. R. Soc. A</i> (2023) — puedes usarla en vez del PLDI si el paper original se siente demasiado denso esta semana.
Repaso opcional de ELBO/inferencia variacional si la Sección 5 no cerró	1 h (opcional)	Ver recomendación de libro en la Sección 5 de este documento.
Escribir el reporte semanal 2	1 h	

Total aprox.: 7.5–9 h (usa la parte opcional de ELBO solo si la sientes necesaria — puedes recuperarla en la Semana 4/Fase B si prefieres avanzar más rápido ahora).

Pregunta guía para cerrar la semana: en tus palabras, ¿por qué DreamCoder necesita *fantasías* (programas sampleados de la librería) y no le basta con reentrenar sobre los programas ya encontrados (*replays*)? (Pista: Sección 3.3, "El rol de las fantasías").

Semana 3 — Composicionalidad: definiciones, benchmarks, y la conexión con DreamCoder

Objetivo de la semana: tener claro qué significa "generalización composicional" en el sentido preciso que usa la literatura (no solo intuitivamente), y empezar a ver dónde DreamCoder calza y dónde no.

Tarea	Tiempo aprox.	Detalle
Documento 1, Sección 4.3 (Definición 4.5 y las 5 propiedades de Hupkes et al.)	1 h	Relee con calma. Esta es <i>la</i> definición formal que vas a usar todo el semestre.
Documento 3, Sección 4.4 ("Connection to compositional generalisation") y Sección 8 (tabla de mapeo a conceptos del curso)	0.5 h	Es corta pero densa: es literalmente el puente entre tus dos temas.
Leer: Lake & Baroni (2018) , " <i>Generalization without systematicity...</i> " (paper de SCAN), ICML	1.5 h	Es el paper que define el benchmark que aparece citado en ambos documentos. Entender SCAN a fondo te da un caso concreto para pensar todo lo demás.
Leer (al menos la introducción y la definición del framework): Hupkes, Dankers, Mul & Bruni (2020) , " <i>Compositionality decomposed</i> ", JAIR	1.5–2 h	Es largo; no hace falta leerlo entero esta semana, pero sí entender las 5 propiedades (sistematicidad, productividad, sustituibilidad, localidad, sobregeneralización) con ejemplos propios.
Lectura complementaria (opcional pero recomendada): Lake & Baroni (2023) , " <i>Human-like systematic generalization through a meta-learning neural network</i> ", Nature	1–1.5 h (opcional)	No es wake-sleep, pero es la referencia más importante de generalización composicional lograda <i>sin</i> biblioteca de programas (vía meta-aprendizaje). Te sirve como contraste: dos caminos muy distintos hacia el mismo objetivo. Útil para tu futura sección de "trabajo relacionado".
Empezar una tabla propia: "mecanismo → qué tipo de composicionalidad ataca → evidencia empírica"	1 h	Ver plantilla sugerida en la Sección 7. Esto es la semilla de tu revisión de literatura de la Fase C.
Escribir el reporte semanal 3	1 h	

Total aprox.: 7.5–9 h

Pregunta guía para cerrar la semana (y para tu reunión): DreamCoder logra que tareas nuevas se resuelvan *recombinando* primitivos de la librería (Sección 4.4, Doc. 3). Pero los benchmarks de la Sección 3 del Documento 1 (SCAN, COGS, etc.) miden composicionalidad de una forma específica (splits que separan combinaciones vistas/no vistas). ¿DreamCoder ha sido evaluado alguna vez en un benchmark de ese tipo, o su noción de "composicional" es distinta (resolver tareas de síntesis de programas cada vez más difíciles)? Esta pregunta, tal cual, puede ser una buena candidata a "pregunta de investigación" para la Fase D — vale la pena que se la plantees a tu profesor en la primera o segunda reunión.

4. Vista rápida de los 3 documentos que te dio tu profesor

Para que tengas claro el rol de cada uno (esto también puede ir, resumido, en tu primer reporte):

- **Documento 1 (Notas de Aprendizaje Composicional, Clase 16):** el marco teórico general del curso — Kolmogorov/MDL como definición de "inteligencia genuina", y un catálogo de "síntomas" medibles (atajos, OOD, composicionalidad, extrapolación, etc.). Es tu **vocabulario común** y tu fuente de definiciones formales.
 - **Documento 3 (Notas de DreamCoder):** el mecanismo concreto de "sueño/vigilia" que vas a estudiar. Es tu **objeto de estudio central**.
 - **Documento 2 (paper de DreamerV3, Hafner et al. 2024):** este es un poco distinto — es aprendizaje por refuerzo basado en modelos del mundo, no wake-sleep Bayesiano. Su relevancia probable es **temática, no técnica**: comparte la metáfora de "soñar" (Dreamer *imagina* trayectorias futuras con su modelo del mundo para entrenar al actor/crítico, igual que DreamCoder *sueña* fantasías para entrenar su modelo de reconocimiento). Te lo recomiendo leer de forma **liviana** en la Semana 1 o 2 (abstract, introducción, y la Figura 3 con su explicación — no necesitas el detalle de RSSM, symlog, etc.); sirve para que en tu informe puedas decir con propiedad "existen (al menos) dos familias de mecanismos de entrenamiento inspirados en 'soñar': imaginación en modelos del mundo (RL) vs. wake-sleep Bayesiano de programas (DreamCoder), y son técnicamente distintos aunque compartan el nombre". Si tu profesor confirma que quiere que compares ambos en serio, eso se planifica en la Fase C, no ahora.
-

5. Libros recomendados (con qué leer de cada uno y por qué)

No necesitas comprar/leer ninguno de estos completos. Están ordenados por prioridad para tu tema.

5.1 Kolmogorov, MDL y computabilidad (prioridad alta)

Li, M. & Vitányi, P. (2008). *An Introduction to Kolmogorov Complexity and Its Applications*, 3ª ed. Springer. — Es la referencia que el Documento 1 cita directamente para casi todos sus teoremas de la Sección 2. Ya que vas a citarlo de todas formas, úsalo como tu libro principal:

- **Capítulo 1 ("Preliminaries"):** aquí está *todo* el repaso de computabilidad/máquinas de Turing que necesitas — justo lo suficiente para entender qué es una máquina universal y por qué K no es computable, sin tener que leer un libro entero de teoría de la computación aparte. **Esta es la ruta recomendada** para el punto de "Máquinas de Turing" que mencionabas.
- **Capítulo 2 ("Algorithmic Complexity"):** definiciones formales, teorema de invarianza, no-computabilidad — el respaldo completo de lo que el Documento 1 resume en su Sección 2.1.
- Hay secciones más adelante dedicadas a MDL ("Minimum Description Length") — útiles en la Fase B si quieres profundizar más allá de Grünwald.

Alternativa si Li & Vitányi se siente demasiado denso como primer contacto con computabilidad: **Sipser, M. (2012). *Introduction to the Theory of Computation*, 3ª ed.** — Capítulos 3 (máquinas de Turing) y 4 (decidibilidad, problema de la parada). Es más pedagógico pero es un desvío más largo. Úsalo solo si te atorras con el Capítulo 1 de Li & Vitányi.

Grünwald, P. D. (2007). *The Minimum Description Length Principle*. MIT Press. — El libro de referencia sobre MDL (más allá del paper original de Rissanen 1978 que ya cita el Documento 1). No lo necesitas completo: **Capítulo 1** (introducción, MDL "crudo" vs. "refinado") te da todo el contexto adicional que puedas necesitar para entender por qué DreamCoder plantea su objetivo como en la Ecuación (1) de esas notas.

5.2 Inferencia variacional / wake-sleep moderno (prioridad media)

Murphy, K. P. (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press. (gratis online: probml.github.io/book2) — Tiene capítulos actualizados sobre inferencia variacional y autoencoders variacionales (VAE), que es exactamente el "primo moderno" de wake-sleep según la Sección 5 del Documento 3. Recomendado por sobre Bishop (más abajo) porque cubre VAEs directamente, que Bishop (2006) no alcanza a cubrir por ser anterior. — Lee solo lo necesario para entender la Ecuación (13) del Documento 3 (el ELBO) con soltura: el capítulo de "Variational Inference" y, si quieres, el de VAEs.

Alternativa/complemento clásico: **Bishop, C. (2006). *Pattern Recognition and Machine Learning*.** Capítulo 10 (Approximate Inference) — muy usado en cursos de grado, buena introducción "de toda la vida" a inferencia variacional, aunque no cubre VAEs (el libro es de antes de 2013).

Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press. (gratis online: deeplearningbook.org) — Como ya vienes de un curso de Deep Learning, este te va a resultar rápido de navegar. El Capítulo 20 ("Deep Generative Models") sitúa históricamente las máquinas de Helmholtz y el algoritmo wake-sleep como antecesores directos de los modelos generativos modernos (VAE, GANs). Útil sobre todo como contexto histórico/narrativo para tu informe.

5.3 PAC learning (repaso, ya lo conoces)

Shalev-Shwartz, S. & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. — Ya lo tienes de tu curso previo. Solo necesitas refrescar el **Capítulo de VC-dimension** para que el Teorema 4.10 del Documento 1 (complejidad de muestra vía VC) no te tome por sorpresa. 20–30 min de repaso, no una lectura nueva.

5.4 Opcionales (solo si te interesa profundizar, no urgente)

- **Gulwani, Polozov & Singh (2017). *Program Synthesis***. Foundations and Trends in Programming Languages (gratis, es un survey en formato libro). Útil si quieres entender mejor el campo de síntesis de programas en general, más allá de DreamCoder.
- **Pierce, B. (2002). *Types and Programming Languages***. Solo si el lambda cálculo tipado de la Sección 4.1 del Documento 3 (gramáticas tipadas) te resulta genuinamente opaco. Las notas ya son bastante autocontenidas en esto — probablemente no lo necesites.

6. Plantilla para tu reporte semanal

Tu profesor pidió explícitamente: máximo 2–3 páginas, con (a) resumen/avances, (b) dudas, (c) propuesta próxima semana. Una plantilla que te puede servir:

markdown

Reporte semana N – [tu nombre]

Fecha:

(a) Qué leí / avancé esta semana

- [Documento/paper 1]: 2-3 líneas de qué entendiste, en tus palabras (no resume la fuente, sintetiza lo que TÚ te llevas).
- [Documento/paper 2]: *ídem.*
- (Opcional) Una idea, conexión o intuición nueva que se te ocurrió.

(b) Dudas

- Duda concreta 1 (idealmente: "entiendo X, pero no veo por qué Y implica Z" – las dudas específicas se resuelven mucho más rápido en la reunión que las genéricas tipo "no entendí la sección 3").
- Duda concreta 2.

(c) Propuesta próxima semana

- Qué piensas leer/hacer.
- Por qué (cómo se conecta con lo anterior).

Consejo sobre "sacarse de bloqueos de forma independiente": cuando te atores en una demostración o definición, ponte un límite de tiempo (30-40 min). Si en ese tiempo no avanzas, anótalo como duda concreta y sigue adelante con el resto de la lectura — no dejes que un solo párrafo bloquee toda la sesión de estudio. Para desbloquearte solo, además de las fuentes de este documento, funcionan bien: buscar el término específico + "intuition" o "explained" en video (hay buenas explicaciones de complejidad de Kolmogorov y de ELBO en YouTube), o leer la sección equivalente en una fuente distinta (a veces la segunda explicación de la misma idea, con otras palabras, es la que hace "clic").

7. Plantilla para tu futura revisión de literatura (Fase C, semanas 7-9)

Puedes ir alimentando esta tabla desde ya, cada vez que encuentres un paper relevante:

Mecanismo	Referencia	¿Qué noción de "sueño/vigilia" usa?	¿Qué noción de composicionalidad ataca?	Evidencia empírica
Wake-sleep Bayesiano de programas	Ellis et al. 2021 (DreamCoder)	Fases wake/abstraction/dreaming, biblioteca como prior aprendido	Composición de primitivos reutilizables (Sec. 4.4)	List processing, Logo, ARC (vía PeARL)
Meta-aprendizaje para composicionalidad	Lake & Baroni 2023 (MLC)	No es wake-sleep; entrenamiento sobre distribución dinámica de tareas	Sistematicidad en el sentido de SCAN	Comparación directa con humanos
...	...	...	...	...

Ya tienes dos candidatos con los que partir esta tabla desde la Semana 3 (ver arriba). En la Fase C la idea es llegar a 8-12 filas, con al menos 2-3 papers de 2023-2025 que no estén ya citados en los documentos de tu profesor (esa búsqueda activa —vía Google Scholar / Semantic Scholar/ arXiv, con términos como *"library learning compositional generalization"*, *"wake-sleep neural network 2024"*, *"neurosymbolic compositional generalization"*— es justamente el ejercicio de "aprender a organizarse y sacarse de bloqueos" que menciona tu profesor: nadie te va a dar esa lista armada).

8. Notas finales

- **No hay una sola forma "correcta" de leer los papers densos.** En la primera pasada de cualquier paper/notas nuevas, tu objetivo es el mapa, no el territorio: qué problema resuelven, qué es lo nuevo, cómo se estructura el argumento. Las demostraciones y detalles se trabajan en la segunda pasada, con lápiz.
- **Prioriza entender por sobre cubrir.** Si en la Semana 2 solo alcanzas a leer bien Hinton et al. (1995) y las Secciones 3-4 de las notas de DreamCoder, pero de verdad las entiendes, vas mejor que si "pasaste" por seis papers sin quedarte con nada. Ajusta el plan a tu ritmo real y dilo en tu reporte (c): es información útil para tu profesor, no una falla tuya.
- **Guarda tus propias notas/glosario desde la Semana 1** (K(x), MDL, PCFG, ELBO, wake-sleep, biblioteca, sistematicidad, productividad...). Te va a ahorrar mucho tiempo en las Fases C-D, cuando necesites moverte rápido entre fuentes nuevas que asumen ese vocabulario.
- Si para la Semana 3 sientes que vas atrasado con el plan, no hay drama: comunícalo en tu reporte y ajusta. El objetivo de este documento es darte dirección, no una carrera contra el reloj.

¡Éxito con el semestre!